

Gate review report — sample (anonymized)

Mode: review
Generated: 2026-10-08 14:22 UTC
Duration: 12m 41s (761s)
Gate confidence: 78%
Commit go/no-go: no-go
Push go/no-go: no-go

[!NOTE] **Sample only — fictional product.** Paths and names below are anonymized (AcmeLedger). No customer IP, secrets, or real API keys appear here.

Audit confidence & gate interpretation
Gate confidence reflects review-package adequacy (checklist coverage + open severity penalties), **not** a “% secure” score. A clean scanner pass with unanswered checklist items still lowers confidence. See RepoLens docs/faq.md → *What do report metrics mean?*

Two-Lane: Fast Brain: 186 file(s) in 0.4s · Slow Brain: 120 file(s) in 698.2s · 12m 41s · 0 critical · 1 high · 0 medium · 1 low

Gate verdict

- **Gate confidence:** 78% (review-package adequacy — not “% secure”)
- **Counts:** Critical 0 · High 1 · Medium 0 · Low 1

Gate and AI audit

DETERMINISTIC GATE: FAIL

- Scanners: 5/5 · Fast Brain Critical/High 1

AI DEEP AUDIT: 82%

- Security 78% · Reliability 95% · Architecture 88%

The gate uses scanners and Fast Brain only. The AI audit is scored separately and never changes the gate verdict.

Metrics

Gate = adequacy of *this review package* for a go/no-go style decision — **not** “% secure” or an architecture grade.

Metric	Value	Meaning
Gate confidence	78%	Lowest band, then a penalty for open Critical/High. Not “% secure.”
Critical	0	Scanner and Fast Brain findings at Critical
High	1	Scanner and Fast Brain findings at High

Metric	Value	Meaning
Medium	0	Do not change the percentages
Low	1	Do not change the percentages
Duration	12m 41s	Wall-clock time for this review
Fast Brain files	186	Whole-tree heuristics
LLM pack files	120	Files sent to the model
Security audit confidence	78%	Security checklist + Critical/High security findings
Reliability audit confidence	95%	Reliability checklist + Critical/High reliability findings
Architecture audit confidence	88%	Architecture checklist + Critical/High architecture findings

How these % are calculated

- **Gate** is the lowest band, then a penalty for missed checklist ids and open C/H.
- Medium and Low findings do not change these percentages.
- Full arithmetic: RepoLens docs/faq.md → *What do report metrics mean?*

P1 — Security

HIGH — Webhook HMAC comparison uses == (timing-leak risk)

Field	Value
Priority	P1
Fingerprint	a1b2c3d4e5f67890
Occurrence	occ-sample-001
Source	llm (verified location)
Location	src/acmeledger/webhooks/stripe_receiver.py:88 (verified → SARIF-eligible)

Explanation. The Stripe webhook handler compares the computed HMAC digest to the Stripe-Signature header with Python's `==`. That comparison is not constant-time, so an attacker who can measure response latency may recover the signing secret byte by byte on a noisy-but-reachable edge.

Impact. Forged webhook events can mark invoices paid, create phantom refunds, or trigger fulfilment without a real charge — direct financial loss and audit-trail gaps.

Recommended fix. Use `hmac.compare_digest` (or Stripe's official SDK verifier) and reject on mismatch before parsing the body.

Code example

```
# Before (timing-variable)
if computed_sig == header_sig:
    handle_event(payload)

# After (constant-time)
import hmac
```

```
if not hmac.compare_digest(computed_sig, header_sig):
    raise UnauthorizedWebhook("bad signature")
handle_event(payload)
```

P2 — Bugs, reliability, performance

No findings in this band.

P3 — Architecture & quality

LOW — Import cycle between billing and notifications

	Field	Value
Priority		P3
Fingerprint		f0e1d2c3b4a59687
Occurrence		occ-sample-002
Source		graph
Location		cycle group: acmeledger.billing ↔ acmeledger.notifications

Explanation. Runtime import graph reports one strongly connected component of size 2. `billing.ledger` imports `notifications.emailer` for receipt sends; `notifications.emailer` imports `billing.ledger` for amount formatting. Cyclicity score contribution: ($2^2 = 4$).

Impact. Hot-reload, testing, and layering suffer; Rule 1 ratchet will fail CI if this score rises above baseline.

Recommended fix. Extract shared money formatting to `acmeledger.money` (or pass plain values into the notifier) so the dependency is one-way: `billing` → `notifications`.

Model notes

The model wrote these. They do not change Critical, High, Medium, or Low.

- Prefer Stripe's `Webhook.construct_event` over hand-rolled HMAC when the SDK is already a dependency.
- Complexity hotspot: `reconcile_day` (`src/acmeledger/billing/reconcile.py:40`) — cyclomatic 18 / cognitive 24; extract day-part helpers before adding tax jurisdictions.

Automated scanners

- **gitleaks:** ran (0 finding(s))
- **semgrep:** ran (0 finding(s))
- **osv:** ran (0 finding(s))
- **trivy:** ran (0 finding(s))
- **checkov:** ran (0 finding(s))

Quality scorecard (Fast Brain)

Signal	Count
Mega-files	0
Deep nesting	1
Near-clone clusters	0
Files scanned	186

Complexity (Fast Brain)

Metric	Value
Functions analysed	412
Issues (above threshold)	9
Max cyclomatic	18
Max cognitive	24

Top complexity hotspots

File	Function	Line	Cyclomatic	Cognitive
src/acmeledger/billing/reconcile.py	reconcile_day	40	18	24
src/acmeledger/webhooks/stripe_receiver.py	dispatch	120	14	19

Import graph

Metric	Value
Status	ok
Packages	1
Modules	64
Cycle groups	1
Cyclicity	4

Deterministic Python import cycles (grimp) — DIP / module-boundary layering signal, not an architecture certification.

Provenance

- **RepoLens:** 0.1.1
- **Git SHA:** deadbeef0123456789abcdef0123456789abcdef (*sample*)
- **Dirty tree:** no
- **Model:** openai / gpt-4.1-mini (*BYOK sample — minutes-scale; not local 32B*)
- **Scanners:** gitleaks, semgrep, osv, trivy, checkov
- **LLM bypassed:** no

Plan to fix

1. **P1:** Replace == HMAC compare with hmac.compare_digest (or Stripe SDK) in stripe_receiver.py; add a unit test that rejects a mutated signature.

2. **P3:** Break the billing ↔ notifications import cycle via a shared money helper; re-run repolens baseline set only after cyclicity drops or stays flat.
3. **Complexity:** Split reconcile_day before the next tax-region feature lands.

Durability gaps

- ☐ Add webhook signature tests (happy / bad sig / replay window).
- ☐ Keep CodeQL + Semgrep in CI; RepoLens complements — does not displace — those engines.
- ☐ Two-Lane: Fast Brain sees 186 file(s); LLM sample pool is 120.

Checklist

These questions check security, reliability, and architecture. An answered question points at a finding. A question that does not apply is closed by a fact from this repository. A question that was not answered is unfinished review work.

- **Answered:** 4 · **Does not apply:** 47 · **Not answered:** 1

Answered

- Webhook / callback authenticity (sec.authn_authz). See “Webhook HMAC comparison uses ==” under P1.
- Import layering / DIP (arch.import_cycles). See “Import cycle between billing and notifications” under P3.

Not answered

- Multi-tenant data isolation (sec.tenant_isolation) — inventory shows shared DB schemas; no finding closed this question. **Follow-up:** confirm row-level filters on org_id or mark N/A with evidence.

End of sample. Generate your own with repolens audit --path ./your-repo --out ./reports (or repolens review --preset release).